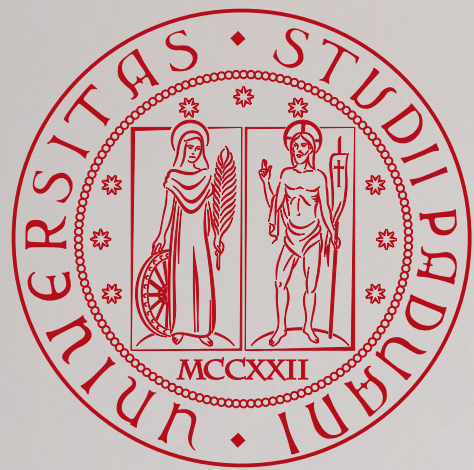
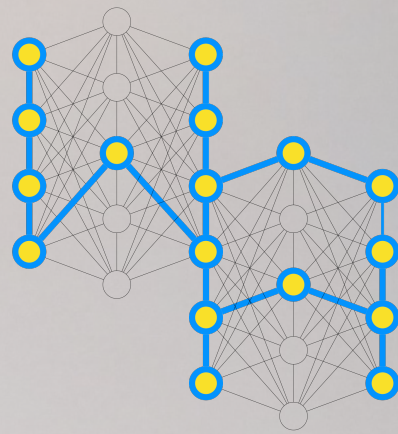




UNIVERSITÀ
DEGLI STUDI
DI PADOVA



JavaScript

Web Applications

Master Degree in Computer Engineering

Master Degree in Cybersecurity

Master Degree in ICT for Internet and Multimedia

Academic Year 2023/2024



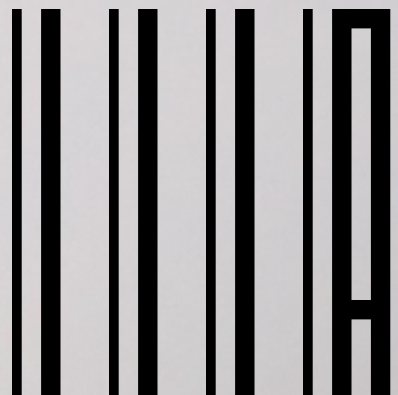
DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

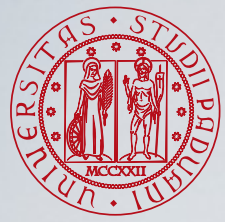
Nicola Ferro

Intelligent Interactive Information Access (IIIA) Hub

Department of Information Engineering

University of Padua





Outline



- Introduction to JavaScript
- JavaScript syntactical rules
- The Document Object Model
- Handling events

Introduction to Javascript

What is JavaScript

- JavaScript is the programming language of the Web, it adds **interactivity** and **custom behaviors** to a Web page.
- Technologies needed to create a Web page:
 - HTML to specify the structure;
 - CSS to specify the presentation;
 - JavaScript to specify the behavior.

HTML

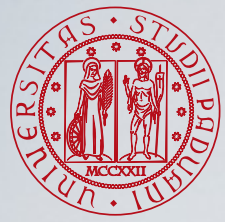


JS



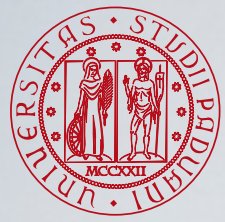
CSS





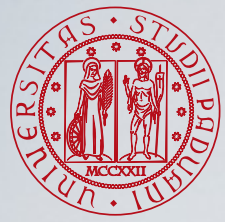
What is JavaScript

- JavaScript is a **high-level, dynamically typed, interpreted** programming language that is well-suited to object-oriented and functional programming styles.
- It is traditionally a **client-side** scripting language, which means it runs on the user's machine and not on the server.
- Nowadays JavaScript is more and more also a server side language (Node.js).



JavaScript vs Java

- “*Java is to Javascript what Car is to Carpet*” — Chris Heilmann
- The name JavaScript is actually somewhat **misleading**: JavaScript is completely different from the Java programming language except for a superficial syntactic resemblance.
- JavaScript was created by Brendan Eich at Netscape in 1995 and originally named LiveScript, but for marketing reasons it was renamed JavaScript or just JS.
- JS bad reputation: for a while it was synonymous with all sorts of dishonest activities: unwanted redirects, unpleasant pop-up windows, and a host of security vulnerabilities.

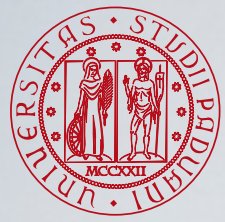


Interaction with Web Pages



How JavaScript makes pages more interactive:

- **Access the content:** select any element, attribute or text from an HTML page;
- **Modify content:** add or remove any element, attribute or text in a HTML page;
- **React to events:** specify that a script should run when a specific event has occurred.

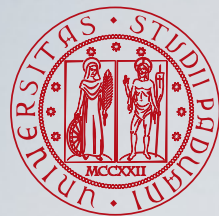


Examples of Interface Logic in JavaScript



What JavaScript can do:

- **Form validation**: altering the contents of the page and blocking the form submission;
- **Slideshow**: display different images within the same space on a given page;
- **Reload part of a page**: request content and information from the server and inject it into the current document as needed, without reloading the entire page;
- **Filtering data**: help users to find the information they need by providing filters;
- **Test for browsers' features and capabilities**: test for the device type and add more user-friendly styles and interaction methods based on the device type.

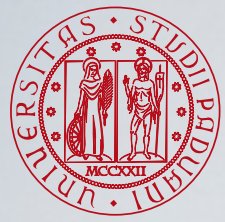


What JavaScript Can't Do



For security reasons browsers impose restrictions on the use of certain JS features that they do support:

- A JavaScript program can open new browser windows, but, to prevent **pop-up** abuse by advertisers, most browsers restrict this feature so that it can happen only in response to a user-initiated event, such as a mouse click.
- A JavaScript program can **close** browser windows that it opened itself, but it is not allowed to close other windows without user confirmation.
- A script **cannot read** or **modify** the content of documents loaded from **other tabs** or **windows**. Similarly, a script cannot register event listeners on pages on different tabs or windows.



Adding JavaScript to a Page



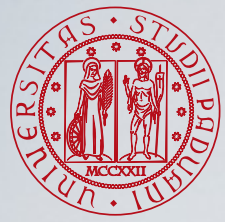
Like CSS, you can embed a script right in a document or keep it in an external file and link it to the page:

- Embedded script:

```
<script>  
... JavaScript code goes here  
</script>
```

- External scripts:

```
<script src="my_script.js"></script>
```

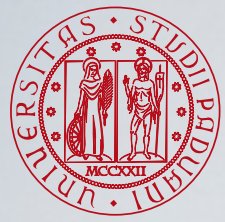



Advantages of External Scripts



There are a number of advantages to using external scripts:

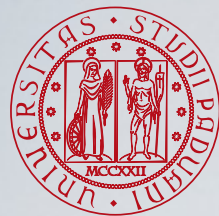
- It **simplifies** the HTML files: it helps keep content and behavior separate.
- When multiple web pages share the same JavaScript code, using the src attribute allows you to maintain only a **single copy** of that code.
- If a file of JavaScript code is shared by more than one page, it only needs to be **downloaded once**.
- Because the src attribute takes an arbitrary URL as its value, a JavaScript program or web page from one web server can employ **code exported** by other web servers.



Execution of JavaScript Program



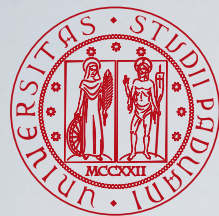
- **JavaScript program** consists of all the JavaScript code in a web page (embedded and external scripts), which see the same Document object and **they share the same set of global functions and variables.**
 - If a script defines a new global variable or function, that variable or function will be visible to any JavaScript code that runs after the script does.
- JavaScript programs are loaded and executed in the **same order as they appear in the document.**
- If a JavaScript program registers an event handler, i.e. a function, this is invoked and executed when the event occurs.
 - Examples of events are: document loaded, user interactions (clicks, submission of a button form, ...)



Script Placement

- The script element goes anywhere in the document: when the browser comes across a `<script>` element, it stops loading the page and start to load the script.
- The most common places for scripts are in the `<head>` of the document and at the very end of the `<body>`.
- **End of the document**, just before the `</body>` tag: is the preferred placement because **the browser will be done parsing** the document and its DOM structure.
- When you want your **script to do something before** the body completely loads, so putting it in the **head will result in better performance**.

Core JavaScript



Case Sensitivity

- JavaScript is a **case-sensitive language**: this means that language keywords, variables, function names, and other identifiers must always be typed with a consistent capitalization of letters.
- Note that HTML is not case-sensitive.
- Many client-side JavaScript objects and properties have the same names as the HTML tags and attributes they represent. While these tags and attribute names can be typed in any case in HTML, in JavaScript they typically must be all lowercase.

JavaScript supports two styles of comments:

- any text between a `//` and the end of a line is treated as a comment and is ignored by JavaScript.

```
// This is a single-line comment.
```

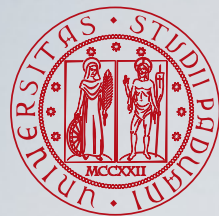
- any text between the characters `/*` and `*/` is also treated as a comment; these comments may span multiple lines but may not be nested.

```
/*
```

```
* This is yet another comment.
```

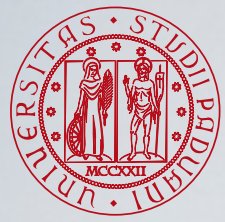
```
* It has multiple lines.
```

```
*/
```

Optional Semicolons

- JavaScript uses the **semicolon** (`;`) to separate statements from each other.
- This is important to make the meaning of your code clear.
- In JavaScript, you can usually omit the semicolon between two statements if those statements are written on separate lines; this might lead to some surprising cases...

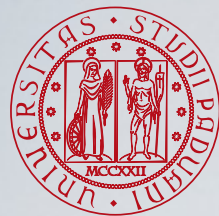


Optional Semicolons



- JavaScript uses the **semicolon (;)** to separate statements from each other.
- This is important to make the meaning of your code clear.
- In JavaScript, you can usually omit the semicolon between two statements if those statements are written on separate lines; this might lead to some surprising cases...

```
var y = x + f
(a+b).toString()
```



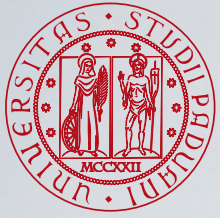
Optional Semicolons



- JavaScript uses the **semicolon** (**;**) to separate statements from each other.
- This is important to make the meaning of your code clear.
- In JavaScript, you can usually omit the semicolon between two statements if those statements are written on separate lines; this might lead to some surprising cases...

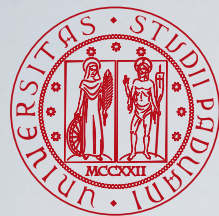
```
var y = x + f
(a+b).toString()
```

```
var y = x + f(a+b).toString()
```

JavaScript Data Types

- JavaScript **types** can be divided into two categories:
 - **primitive types**: numbers, strings of text, and Boolean;
 - **object types** (e.g. array, function, ...).
- Special JavaScript values **null** and **undefined** are primitive values, but they are not numbers, strings, or booleans.
- Any JavaScript value that is not a number, a string, a boolean, or null or undefined is an object.
- The JavaScript interpreter performs automatic garbage collection for memory management.



Variables

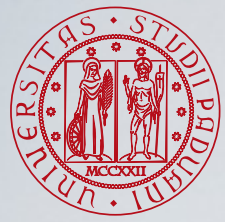
- Before you use a variable in a JavaScript program, you should **declare** it. Variables are declared with the **var** keyword.

```
var i;  
var sum;
```

- You can combine variable declaration with variable initialization:

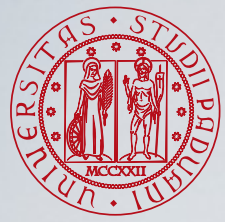
```
var message = "hello";
```

- If you don't specify an initial value for a variable with the var statement, the variable is declared, but its value is undefined until your code stores a value into it.



Null and Undefined

- **null** is a language keyword that is usually used to indicate the **absence of a value**.
- **undefined** value is the value of variables that have **not been initialized** and the value you get when you query the value of an object property or array element that does not exist.
- **null** and **undefined** both indicate an absence of value and can often be used interchangeably.



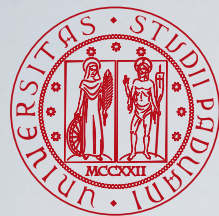
Numbers



- JavaScript does **not** make a **distinction** between integer values and floating-point values. All numbers in JavaScript are represented as floating-point values.

```
var a = 5;  
var pi = 3.14;
```

- Operations: addition (+), subtraction (−), multiplication (*), division (/), and modulo (%).
- More complex mathematical operations with the `Math` object.



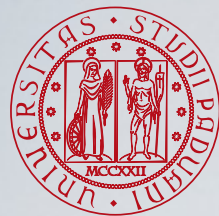
String



- Strings are JavaScript's type for representing **text**.
- JavaScript's strings (and its arrays) use **zero-based indexing**.
- Enclose the characters of the string within a matched pair of single or double quotes (' or ").

```
var a = 'Hello'  
var b = "bye"
```

- The backslash character (\) can be used as escape character.
- Use the **+** operator to concatenate strings.

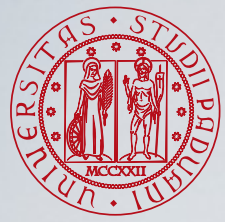


Booleans

- For a boolean there are only two possible values expressed with the reserved words **true** and **false**.

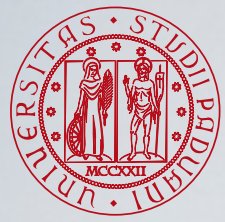
```
var a = true;  
var b = false;
```

- Boolean values have a **toString()** method that you can use to convert them to the strings "true" or "false"
- Boolean operators: AND (&&), OR (||), NOT (!).



JavaScript Statements

- `var`
- `if/else`
- `else if`
- `switch`
- `while`
- `do/while`
- `for`
- `break`
- . . .



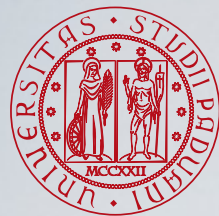
JavaScript Objects

- JavaScript objects are **associative arrays**

- Access a property of an object:

`object.property`
`object[ "property" ]`

- In C++, Java, and similar strongly typed languages, an object can have only a fixed number of properties, and the names of these properties must be defined in advance. In JavaScript this rule does not apply: a program can create any number of properties in any object.



Create a JavaScript Object

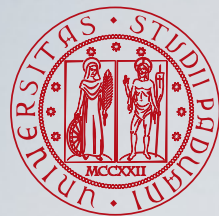


- The easiest way to create an object is to include an **object literal** in your JavaScript code: a comma-separated list of colon-separated name:value pairs and functions, enclosed within curly braces.

```
var o = {  
  data_prop1: value1,  
  data_prop2: value2,  
  method_1() { /* function body here */ },  
  method_2(value) { /* function body here */ }  
};
```

- Otherwise you can define a **constructor** to initialize an object:

```
var a = new Array();  
var d = new Date();
```

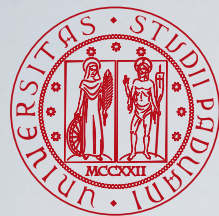



JS Object types

- The example of the previous slide is limited, since it only create a single object
- It is necessary oftentimes to have “blueprints” for creating many instances of objects of the same type
- JS also uses **object constructor functions**
- Objects of the same type can be created by calling the constructor function with the `new` keyword

```
function Person(first, last, age, eye) {  
    this.firstName = first;  
    this.lastName = last;  
    this.age = age;  
    this.eyeColor = eye;  
}
```

```
var myFather = new Person("John", "Doe", 50, "blue");  
var myMother = new Person("Sally", "Rally", 48, "green");
```



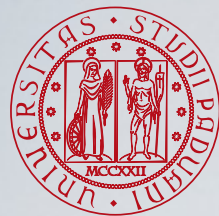
JS Object types



- With a constructor function it is thus possible to set default values for the fields of an object.
- It is also possible to define methods inside the constructor function

```
function Person(first, last, age, eye) {  
  this.firstName = first;  
  this.lastName = last;  
  this.age = age;  
  this.eyeColor = eye;  
  this.plusOne = function() {this.age = this.age + 1};  
  this.name = function() {  
    return this.firstName + " " + this.lastName  
  };  
}
```

```
// Display full name  
document.getElementById("demo").innerHTML =  
"My father is " + myFather.name();
```

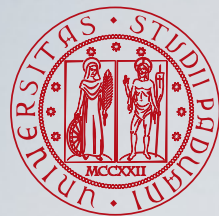


JavaScript Objects: this keyword



```
var person = {  
  firstName: "John",  
  lastName : "Doe",  
  id        : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

- In a function, the `this` keyword refers to the “owner” of the function.
- In the example, `this` is the object `person`, that owns the `fullName` function.
- Thus, `this.firstName` refers to the property `firstName` of the instance of the object



Deleting Properties

```
var book = {  
  title: "JavaScript for Kids",  
  subtitle: "A Playful Introduction to  
  Programming",  
  audience: "children"  
}
```

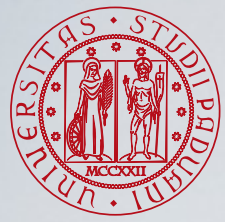
- The delete operator removes a property from an object: not the value of the property, but the property itself.

```
delete book.audience
```

- A delete expression evaluates to true if the delete succeeded or if the delete had no effect.

- JavaScript arrays can have elements of **any type** and different elements of the same array may be of different types.
- JavaScript arrays are **dynamic** and there is no need to declare a fixed size when the array is created.

```
var empty = [];  
var primes = [2, 3, 5, 7, 11];  
var misc = [ 1.1, true, "a", ];  
var nest = [[1, {x:1, y:2}], [2, {x:3, y:4}]];  
var v_0 = new Array();  
var v_1 = new Array(10);  
var v_2 = new Array(5, 4, 3, 2, 1);
```



Access Array Elements and Array Length

- You **access** an element of an array using the `[ ]` operator.

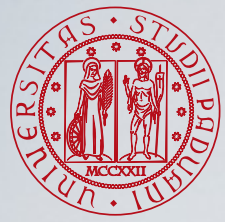
`a[0]`

- Every array has a **length** property which specifies the number of elements in the array.

`a.length`

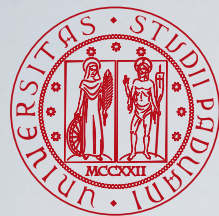
- Iterate** through array elements:

```
for(var i = 0; i < a.length; i++) {  
    . . .  
}
```

Array Methods

- `Array.join()` method converts all the elements of an array to strings and concatenates them, returning the resulting string.
- `Array.reverse()` method reverses the order of the elements of an array and returns the reversed array.
- `Array.sort()` sorts the elements of an array in place and returns the sorted array.
- `Array.concat()` method creates and returns a new array that contains the elements of the original array on which `concat()` was invoked, followed by each of the arguments to `concat()`.
- . . .



ForEach Method

- The **forEach()** method iterates through an array, invoking a function you specify for each element.
- You pass the function as the first argument to `forEach()`, which then invokes your function with three arguments: the **value** of the array element, the **index** of the array element, and the **array** itself.

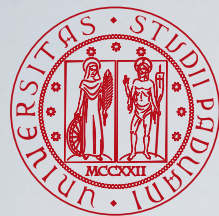
```
var data = [1,2,3,4,5];  
var sum = 0;  
data.forEach(function(value) { sum += value; });  
  
data.forEach(function(v, i, a) { a[i] = v + 1; });
```

- Functions are defined with the `function` keyword followed by:
 - The name of the function;
 - A pair of parentheses which contain the function parameters;
 - A pair of curly braces which contains the body of the function.

```
addNumbers ( ) {  
    return 2 + 2;  
}
```

```
addNumbers ( a , b ) {  
    return a + b;  
}
```

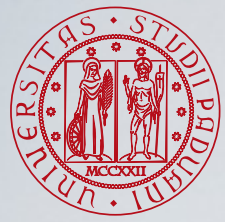

The Window Object



The Window Object

- JavaScript gives you access to and the ability to manipulate the parts of the browser window, known as the **window** object. It represents an open window in a browser.
- The window object has a number of properties and methods:
 - `setTimeout()` and `setInterval()`;
 - `location`;
 - `history`;
 - `alert()`;
 - `confirm()`;
 - `prompt()`;
 - ...

More documentation here: https://www.w3schools.com/jsref/obj_window.asp



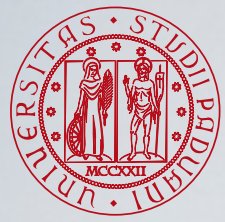
Dialog Boxes



- The Window object provides three methods for displaying simple dialog boxes to the user:
 - **alert()** displays a message to the user and waits for the user to dismiss the dialog.
 - **confirm()** displays a message, waits for the user to click an OK or Cancel button and returns a boolean value.
 - **prompt()** displays a message, waits for the user to enter a string, and returns that string.

```
do {  
    var name = prompt("What is your name?");  
    var correct = confirm("You entered '" + name +  
        "'.\n" + "Click Okay to proceed or Cancel to re-  
        enter.");  
} while (!correct)  
alert("Hello, " + name);
```


- `setTimeout()` and `setInterval()` allow you to register a function to be invoked once or repeatedly after a specified amount of time has elapsed.
- `setTimeout()`: method schedules a function to run after a specified number of milliseconds elapses.
- `setInterval()`: is like `setTimeout()` except that the specified function is invoked repeatedly at intervals of the specified number of milliseconds.
- They both take as input arguments the called function and the interval of time, expressed in milliseconds.

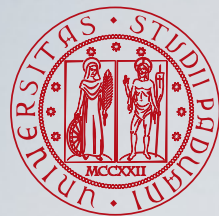


Browser and Screen Information



- Scripts sometimes need to obtain information about the web browser in which they are running or the desktop on which the browser appears.
- The **navigator** property of a `Window` object refers to a `Navigator` object that contains browser vendor and version number information. The `Navigator` object has four properties:
 - **appName**: full name of the web browser;
 - **appVersion**: browser vendor and version information;
 - **userAgent**: string that the browser sends in its User-Agent HTTP header;
 - **platform**: the operating system.
- The **screen** property of a `Window` object refers to a `Screen` object that provides information about the **size** of the user's display and the number of **colors** available on it.

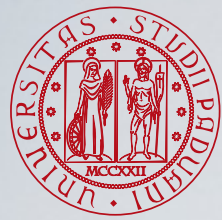
The Console Object



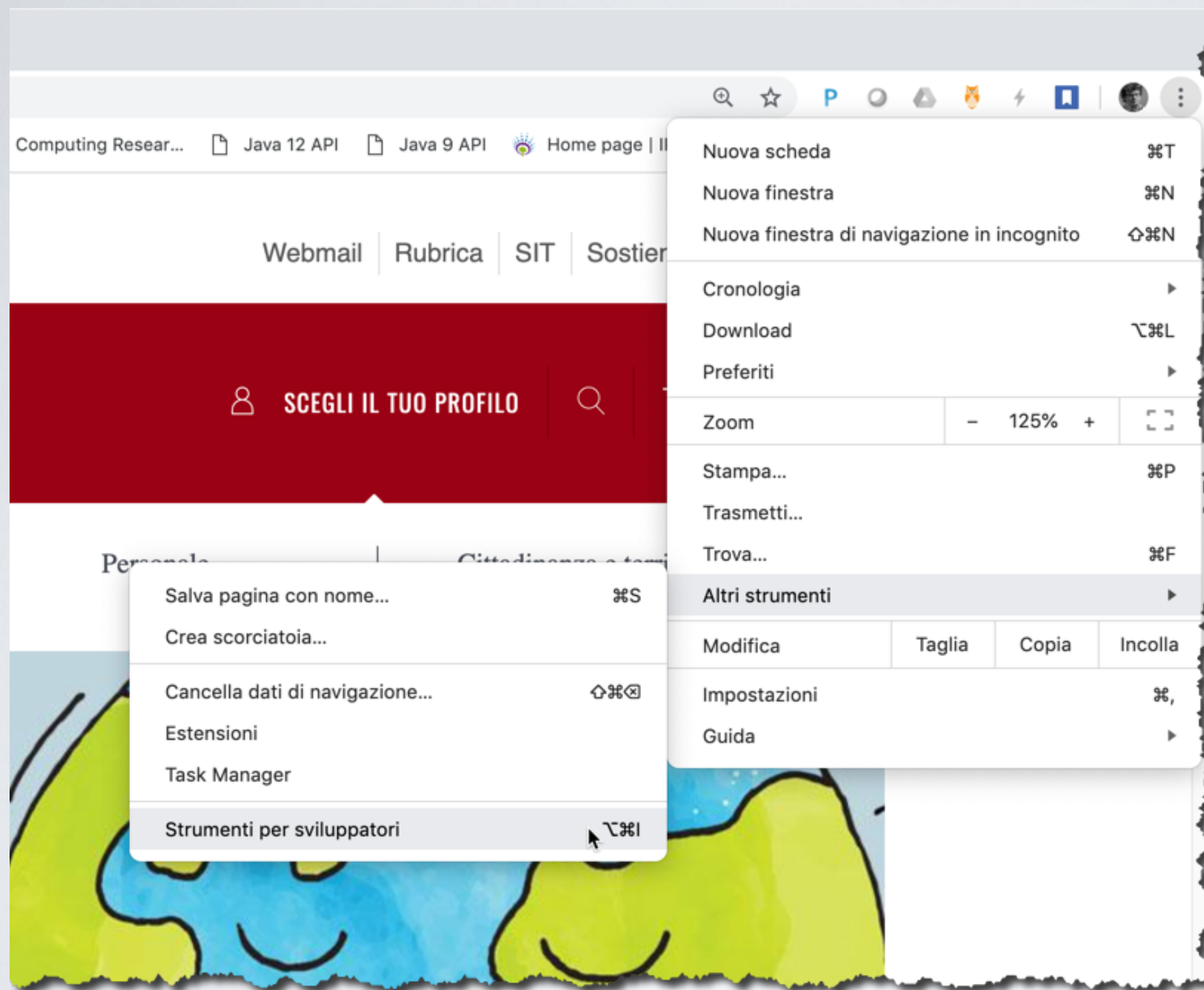
The Console Object

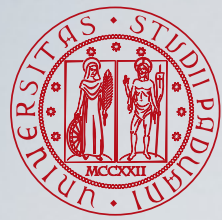


- The **console** object provides access to the browser's debugging console
- The specifics of how it works varies from browser to browser, but there is a de facto set of features that are typically provided
 - `log()`, `trace()`, `debug()`, `info()`, `warn()`, `error()`: output log messages with increasing level of severity, together with additional information, e.g. the stack trace in the case of `trace()`
 - `time()`, `timeEnd()`, `timeLog()`: starts and stops a timer and logs the time passed
 - `assert()`: logs a message and stack trace to console if the first argument is `false`
 - `dir()` logs a JavaScript representation of the specified object. If the object being logged is an HTML element, then the properties of its DOM representation are printed,
 - `table()`: logs an array of objects as a table



Browser Developer Tools: Chrome





Browser Developer Tools: Chrome



Uniweb | Area stampa | Dipartimenti | Scuole

Webmail | Rubrica | SIT | Sostieni Unipd | 800 anni



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

SCEGLI IL TUO PROFILO



IT EN CN

Future matricole

Studentesse e studenti

Personale

Cittadinanza e territorio



Elements Console Sources Network Performance Memory Application Security Audits

```
<!doctype html PUBLIC "-//W3C//DTD XHTML+ARIA 1.0//EN" "http://www.w3.org/MarkUp/DTD/xhtml-aria-1.dtd">
<!--[if IE 7]> <html xmlns="http://www.w3.org/1999/xhtml" class="nojs" lang="it" > <![endif]-->
<!--[if IE 8]> <html xmlns="http://www.w3.org/1999/xhtml" class="nojs" lang="it" > <![endif]-->
<!--[if (gte IE 9)]!(IE)]><!-->
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="it" class="js">
  <!--<![endif]-->
  <head>...</head>
  <body class="eu-cookie-compliance-processed"> == $0
    <div id="skip-link">...</div>
    <!-- header -->
    <div id="header" class="row">...</div>
    <!-- main -->
    <div id="main" class="row header-categories-opened">...</div>
    <!-- footer -->
    <div id="footer" class="footer">...</div>
    <script type="text/javascript">...</script>
    <script type="text/javascript">...</script>
    <script type="text/javascript" src="https://www.unipd.it/sites/unipd.it/modules/contrib/eu_cookie_compliance/js/eu_cookie_compliance.js?ppdh6z">
    </script>
    <script type="text/javascript" src="/sites/unipd.it/themes/unipd_2017/js/jquery-3.1.1.min.js"></script>
    <script type="text/javascript" src="/sites/unipd.it/themes/unipd_2017/js/jquery-migrate-3.0.0.min.js"></script>
    <!-- carousel -->
    <script type="text/javascript" src="/sites/unipd.it/themes/unipd_2017/js/slick.js"></script>
    <script type="text/javascript" src="/sites/unipd.it/themes/unipd_2017/js/script_2018_11.js"></script>
  </body>
</html>
```

html.js body.eu-cookie-compliance-processed

Styles Computed Event Listeners DOM Breakpoints Properties Accessibility

Filter :hov .cls +

```
element.style {
}

body {
  width: 100%;
  margin: 0;
  padding: 0;
  overflow-x: hidden;
}

body {
  margin: 0;
}

* {
  -webkit-box-sizing: border-box;
  -moz-box-sizing: border-box;
  box-sizing: border-box;
}

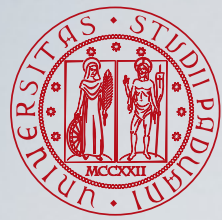
body {
  display: block;
  margin: 8px;
}

Inherited from html.js

html {
  font-size: 1em;
  line-height: 1.4;
}

html, button, input, select, textarea {
  color: #222;
}

html, button, input, select, textarea {
```

Browser Developer Tools: Chrome



Uniweb | Area stampa | Dipartimenti | Scuole | Webmail | Rubrica | SIT | Sostieni Unipd | 800 anni

Computing Resear... | Java 12 API | Home page | Il

UNIVERSITÀ DEGLI STUDI DI PADOVA

SCEGLI IL TUO PROFILO

IT EN CN

Future matricole | Studentesse e studenti | Personale | Cittadinanza e territorio

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Audits

Download | Preferiti | Zoom | Stampa... | Trasmetti... | Trova... | Altri strumenti

Styles | Computed | Event Listeners | DOM Breakpoints | Properties | Accessibility

Filter

element.style {

body {

width: 100%;

margin: 0;

padding: 0;

overflow-x: hidden;

body {

margin: 0;

* {

-webkit-box-sizing: border-box;

Uniweb | Area stampa | Dipartimenti | Scuole | Webmail | Rubrica | SIT | Sostieni Unipd | 800 anni

UNIVERSITÀ DEGLI STUDI DI PADOVA

SCEGLI IL TUO PROFILO

IT EN CN

Future matricole | Studentesse e studenti | Personale | Cittadinanza e territorio

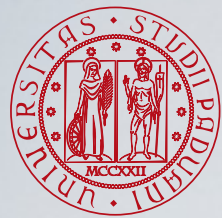
Elements | Console | Sources | Network | Performance | Memory | Application | Security | Audits

top | Filter | Default levels

[Deprecation] chrome.loadTimes() is deprecated, instead use standardized API: nextHopProtocol in Navigation Timing 2. <https://www.chromestatus.com/features/5637885046816768>. content.js:4

[Deprecation] chrome.loadTimes() is deprecated, instead use standardized API: nextHopProtocol in Navigation Timing 2. <https://www.chromestatus.com/features/5637885046816768>. content.js:5

JQMIGRATE: Migrate is installed, version 3.0.0 jquery-migrate-3.0.0.min.js:2



Browser Developer Tools: Chrome



Uniweb | Area stampa | Dipartimenti | Scuole | Webmail | Rubrica | SIT | Sostieni Unipd | 800 anni

Università di Padova

https://www.unipd.it

App | Library Genesis | Sci-Hub: removin... | TREC Washington... | Data-Driven Disco... | Google Cached P... | Computing Resear... | Java 12 API | Java 9 API | Home page | IRIS... | ECIR 2016 Admin

Uniweb | Area stampa | Dipartimenti | Scuole | Webmail | Rubrica | SIT | Sostieni Unipd | 800 anni

UNIVERSITÀ DEGLI STUDI DI PADOVA

SCEGLI IL TUO PROFILO

IT EN CN

Future matricole | Studentesse e studenti | Personale | Cittadinanza e territorio

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Audits

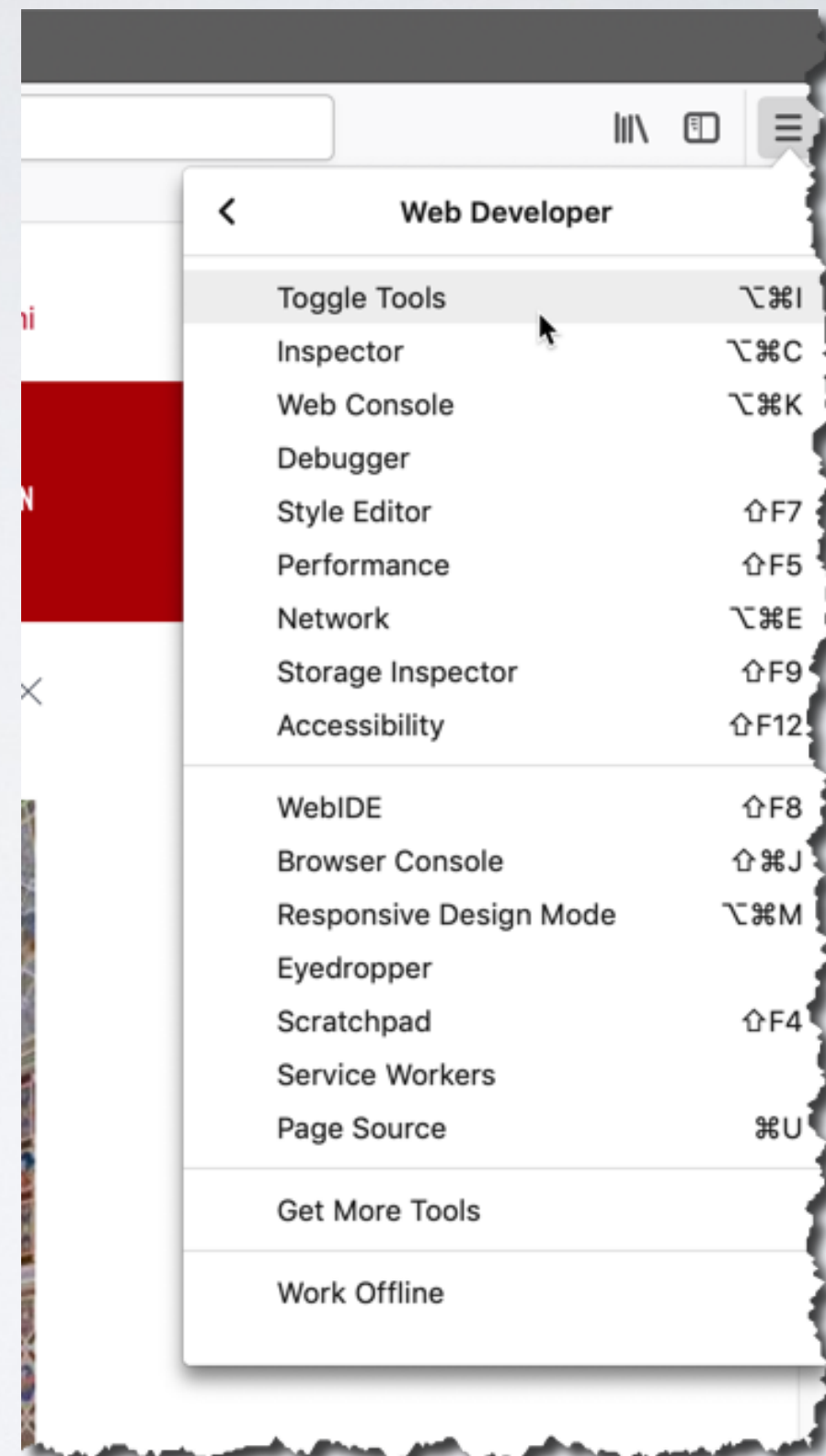
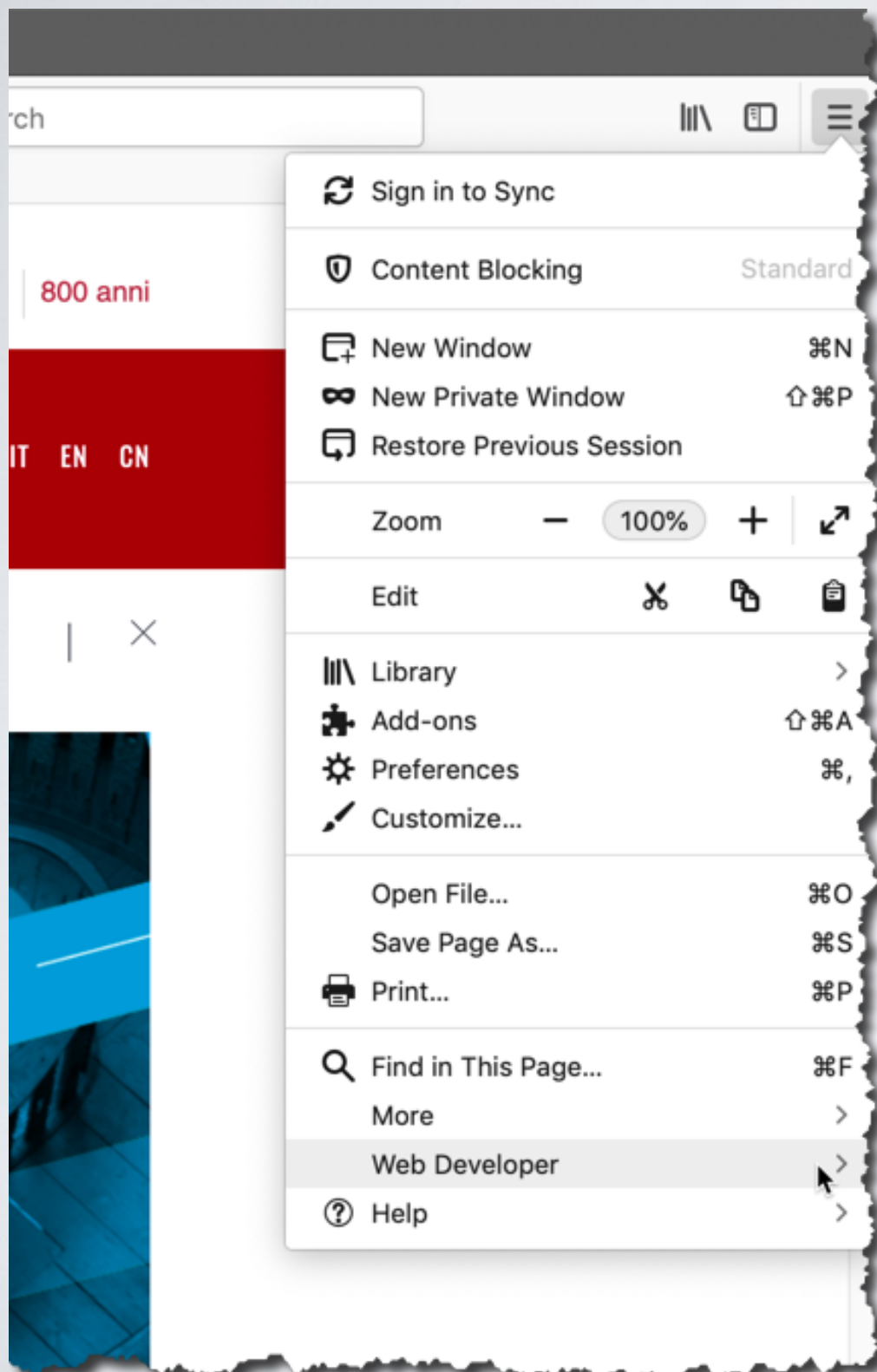
View: [Group by frame] [Preserve log] [Disable cache] [Offline] [Online]

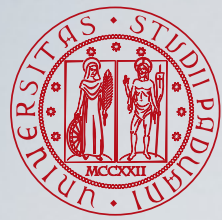
Filter [Hide data URLs] [All] XHR JS CSS Img Media Font Doc WS Manifest Other

Name	Method	Status	Protocol	Type	Initiator	Size	Time	Waterfall
www.unipd.it	GET	200	http/1.1	document	Other	14.4 KB	7.64 s	
system.base.css?ppdh6z	GET	200	http/1.1	stylesheet	(index)	2.3 KB	91 ms	
date.css?ppdh6z	GET	200	http/1.1	stylesheet	(index)	1.6 KB	91 ms	
ctools.css?ppdh6z	GET	200	http/1.1	stylesheet	(index)	723 B	95 ms	
style.css?ppdh6z	GET	200	http/1.1	stylesheet	(index)	28.4 KB	242 ms	
slick.css	GET	200	http/1.1	stylesheet	(index)	1.0 KB	198 ms	
slick-theme.css	GET	200	http/1.1	stylesheet	(index)	1.6 KB	197 ms	
oswald.css	GET	200	http/1.1	stylesheet	(index)	998 B	217 ms	
jquery-1.7.1.min.js?v=1.7.2&v=1.7.2	GET	200	http/1.1	script	(index)	32.8 KB	582 ms	
jquery.once.js?v=1.2	GET	200	http/1.1	script	(index)	1.5 KB	517 ms	
drupal.js?ppdh6z	GET	200	http/1.1	script	(index)	7.2 KB	582 ms	
jquery.cookie-1.4.1.min.js?v=1.4.1	GET	200	http/1.1	script	(index)	1.2 KB	580 ms	
admin_devel.js?ppdh6z	GET	200	http/1.1	script	(index)	973 B	580 ms	
apachesolr_sort.js?ppdh6z	GET	200	http/1.1	script	(index)	835 B	579 ms	
it_Dwd0lvSls5GIDAOJHzctiYd1WIJ6wV5sTVp7Cxfjpa.js?ppdh6z	GET	200	http/1.1	script	(index)	2.0 KB	597 ms	
jwplayer.js?ppdh6z	GET	200	http/1.1	script	(index)	42.2 KB	732 ms	
custom_search.js?ppdh6z	GET	200	http/1.1	script	(index)	1.7 KB	696 ms	
responsive_menus_sldr.js?ppdh6z	GET	200	http/1.1	script	(index)	1.6 KB	701 ms	
jquery.sidr.min.js?ppdh6z	GET	200	http/1.1	script	(index)	2.0 KB	700 ms	
jwplayer.js?ppdh6z	GET	200	http/1.1	script	(index)	42.2 KB	764 ms	

85 requests | 1.5 MB transferred | 2.4 MB resources | Finish: 11.83 s | DOMContentLoaded: 8.75 s | Load: 10.72 s

Browser Developer Tools: Firefox

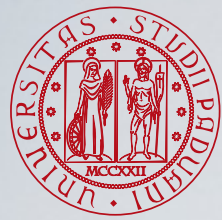




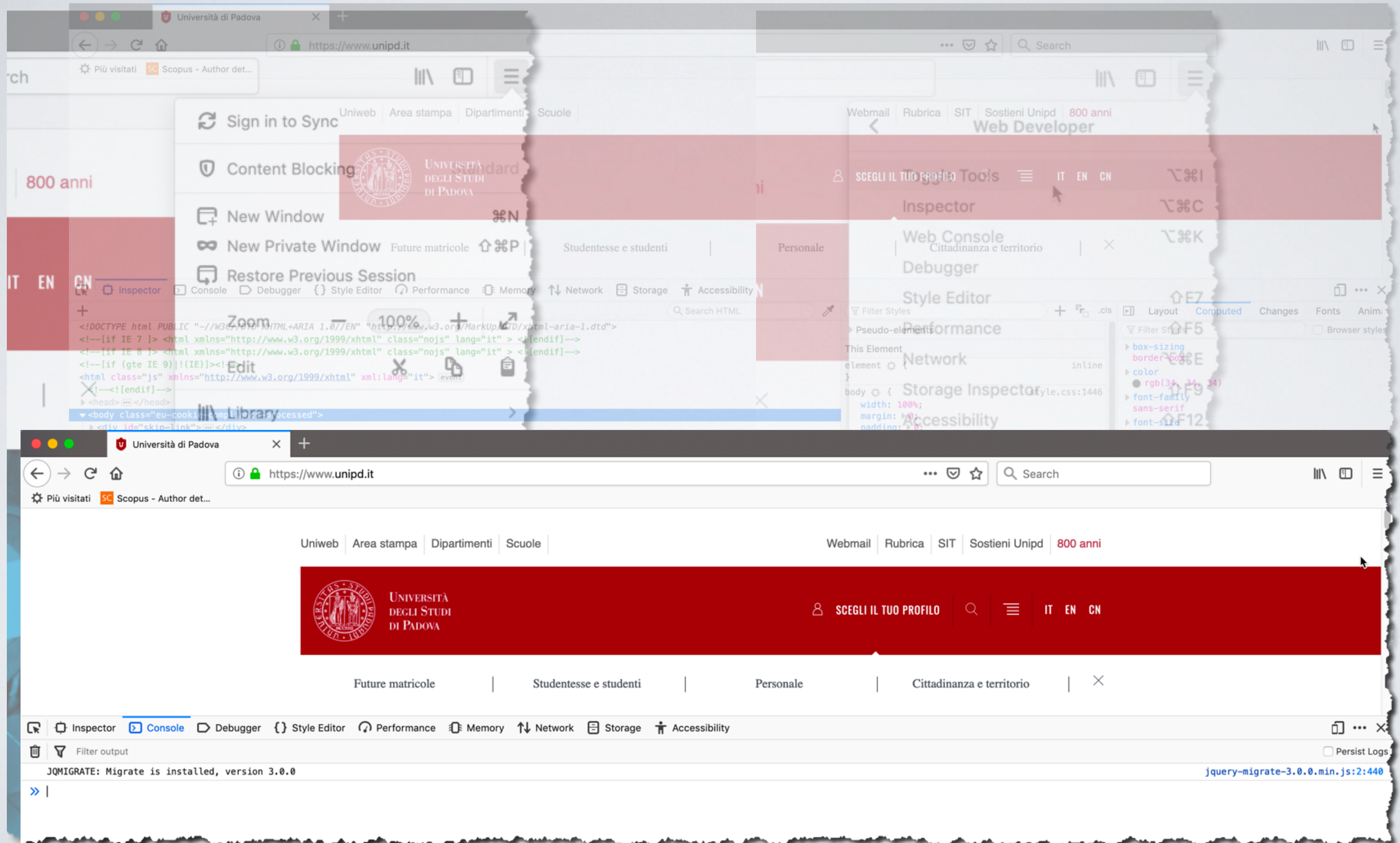
Browser Developer Tools: Firefox

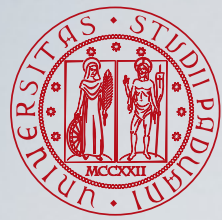


The screenshot displays the Firefox browser with the University of Padua website open. The Developer Tools are open, showing the HTML and CSS of the page. The HTML panel on the left shows the document structure, with the `<body>` element selected. The CSS panel on the right shows the styles applied to the selected element, including `width: 100%`, `margin: 0`, `padding: 0`, and `overflow-x: hidden`. The page header includes navigation links like "Uniweb", "Area stampa", "Dipartimenti", "Scuole", "Webmail", "Rubrica", "SIT", "Sostieni Unipd", and "800 anni". The page also features a red banner with the University of Padua logo and a navigation menu with links like "FUTURE MATRICOLE", "STUDENTESSE E STUDENTI", "PERSONALE", and "CITTADINANZA E TERRITORIO".



Browser Developer Tools: Firefox





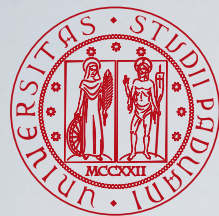
Browser Developer Tools: Firefox



The screenshot displays the Firefox browser with the University of Padua website open. The Developer Tools are open, showing the Network tab with a list of requests. Overlays for 'Save Page As...', 'Print...', 'Find in This Page...', 'Web Developer', 'Scratchpad', 'Service Workers', 'Page Source', 'Cittadinanza e territorio', 'Get More Tools', and 'Web Offline' are visible.

Status	Method	Domain	File	Protocol	Cause	Type	Transferred	Size	0 ms	160 ms	320 ms	479 ms	640 ms	800 ms	960 ms
200	GET	www.unipd.it	slick.css	HTTP/1.1	stylesheet	css	1.04 KB	1.63 KB	116 ms						
200	GET	www.unipd.it	slick-theme.css	HTTP/1.1	stylesheet	css	1.61 KB	3.92 KB		96 ms					
200	GET	www.unipd.it	oswald.css	HTTP/1.1	stylesheet	css	0.98 KB	1.93 KB						98 ms	

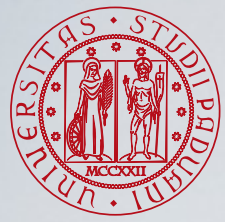
The Document Object Model



The DOM



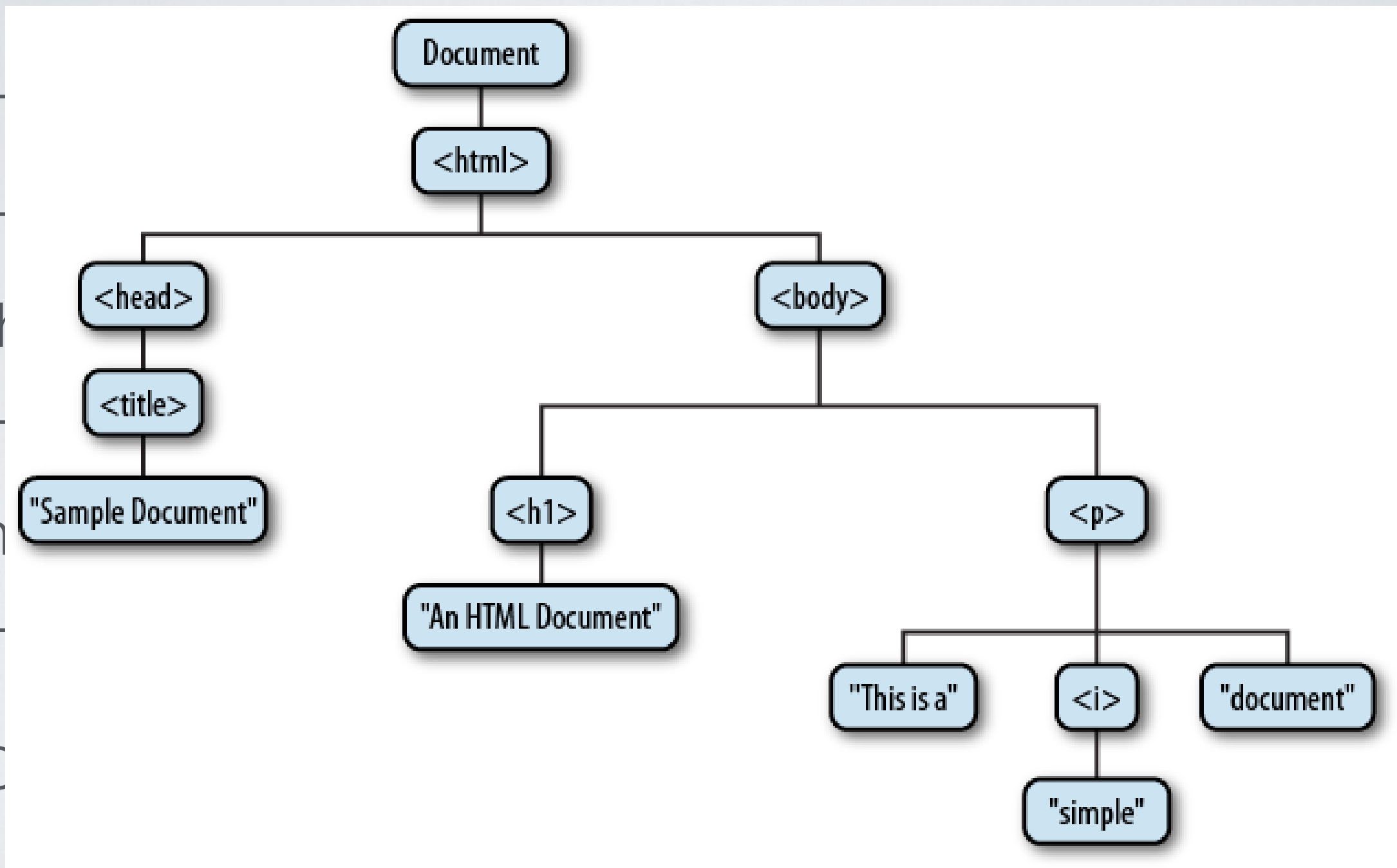
- Every `Window` object has a `document` property that refers to a `Document` object, which represents the content of the window.
- The `Document` object is part of the **Document Object Model**, or **DOM**, which is the fundamental API for representing and manipulating the content of HTML.
- Recall: tree representation of an HTML document contains nodes representing HTML elements.

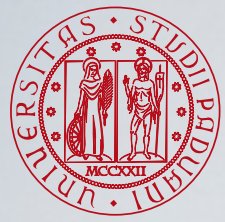


DOM Representation of a Document



- The DOM represents the HTML document as a tree.
- The root of the tree is the Document node that represents the entire document.
- The nodes that represent HTML elements are Element nodes.
- The nodes that represent text are Text nodes.
- Document, Element, and Text are subclasses of Node.

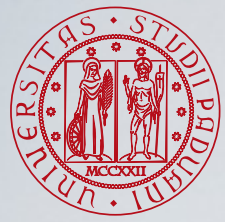




DOM Representation of a Document



- The DOM represents the HTML document as a tree.
- The root of the tree is the Document node that represents the entire document.
- The nodes that represent HTML elements are Element nodes.
- The nodes that represent text are Text nodes.
- Document, Element, and Text are subclasses of Node.

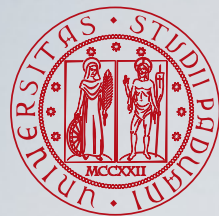


Selecting Document Elements



The DOM defines a number of ways to select elements:

- with a specified **id** attribute;
- with a specified **name** attribute;
- with the specified **tag** name;
- with the specified **CSS class** or classes;
- matching the specified **CSS selector**



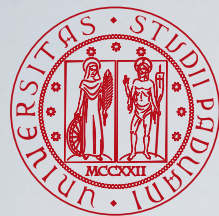
Selecting Elements by Id



- Recall: any HTML element can have an `id` attribute and its value must be unique within the document.
- You can select an element based on this unique `id` with the `getElementById()` method of the Document object.

```
var section1 = document.getElementById("section1");
```

- This is the simplest and most commonly used way to select elements.

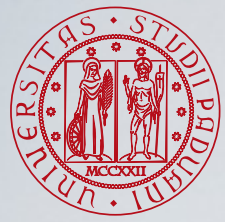


Selecting Elements by Name

- Recall: the HTML `name` attribute is intended to assign names to form elements, and the value of this attribute is used when form data is submitted to a server. Unlike `id`, however, the value of a name attribute does not have to be unique: multiple elements may have the same name (radio buttons and checkboxes).
- To select HTML elements based on the value of their name attributes, you can use the `getElementsByName()` method of the Document object:

```
var radiobuttons = document.getElementsByName("favorite_color");
```

- It returns a NodeList object that behaves like a read-only array of Element objects.

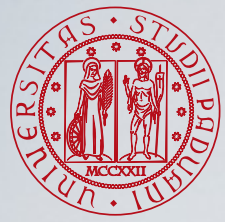


Selecting Elements by Type

- You can select all HTML elements of a specified type (or tag name) using the `getElementsByTagName()` method of the Document object.

```
var spans = document.getElementsByTagName("span");
```

- Like `getElementById()`, `getElementsByTagName()` returns a NodeList object.



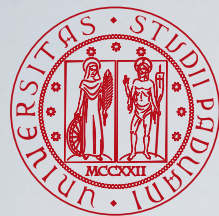
Selecting Elements by CSS Class and Selectors

- You can select all HTML elements of a specified class using the `getElementsByClassName()` method

```
var warnings = log.getElementsByClassName("warning");
```

- `querySelectorAll()` allows you to access nodes of the DOM based on a CSS-style selector.

```
var sidebarPara = document.querySelectorAll(".sidebar p");  
var textInput = document.querySelectorAll("input[type='text']");
```

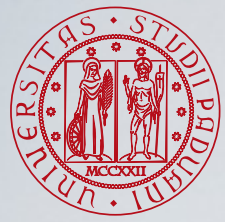


Node Object: Properties



The Document object, its Element objects, and the Text objects are all Node objects, which have the following properties:

- `parentNode`
- `childNodes`
- `firstChild`, `lastChild`
- `nextSibling`, `previousSibling`
- `nodeType`: the kind of node this is. Document nodes have the value 9. Element nodes have the value 1. Text nodes have the value 3. Comments nodes are 8 and Document-Fragment nodes are 11.
- `nodeValue`: the textual content of a Text or Comment node.
- `nodeName`: the tag name of an Element, converted to uppercase.

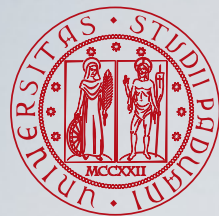


Document as a Tree of Element



When you are primarily interested in the Elements of a document instead of the text within them, you can treat a document as a tree of Element objects, ignoring Text and Comment nodes. Element properties are:

- `children`, returns only Element objects.
- `firstElementChild`, `lastElementChild`
- `nextElementSibling`, `previousElementSibling`
- `childElementCount`: the number of element children.



Attributes as Element Properties



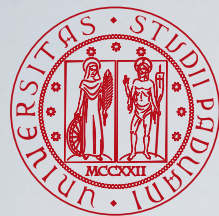
- The HTMLElement objects define read/write **properties** that mirror the HTML **attributes** of the elements.
- Example: to query the URL of an image, you can use the **src** property of the HTMLElement that represents the `<img>` element:

```
var image = document.getElementById( "myimage" );  
var imgurl = image.src;
```

- The Element type also defines **getAttribute()** methods that you can use to query HTML attributes:

```
var image = document.getElementById( "myimage" );  
var imgurl = image.getAttribute( "src" );
```

- Attribute values are all treated as strings, this means that **getAttribute()** never returns a number, boolean, or object.

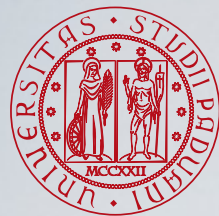


Other Element Methods



- `setAttribute()` methods that you can use to set HTML attributes.
- `hasAttribute()` checks for the presence of a named attribute.
- `removeAttribute()` removes an attribute entirely.

```
var image = document.getElementById("myimage");  
image.setAttribute("src", "newimage.jpg");
```

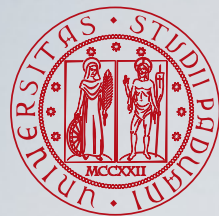


Manipulating Nodes



It is possible to alter a document at the level of individual nodes. The Document type defines methods for **creating Element** and **Text** objects, and the Node type defines methods for **inserting**, **deleting**, and **replacing nodes** in the tree.

```
function loadasync(url) {  
    var head = document.getElementsByTagName("head")[0];  
    var s = document.createElement("script");  
    s.src = url;  
    head.appendChild(s);  
}
```

Creating Nodes



- You can create new Element nodes with the `createElement()` method of the Document object. Pass the tag name of the element as the method argument:

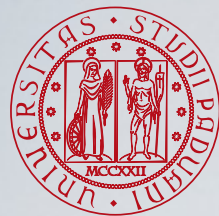
```
var newDiv = document.createElement("div");
```

- Text nodes are created with a similar method:

```
var ourText = document.createTextNode("Put text here.");
```

- Once you create an element in this way, that new element remains “floating” until you add it to the document.

- Once you have a new node, you can insert it into the document.
- **appendChild()** is invoked on the Element node that you want to insert into, and it inserts the specified node so that it becomes the **last child** of that node.
- **insertBefore()** is like appendChild(), but it takes two arguments: the first argument is the node to be inserted, the second argument is the node before which that node is to be inserted. This method is invoked on the node that will be the parent of the new node, and the second argument must be a child of that parent node.



Inserting Nodes: Examples

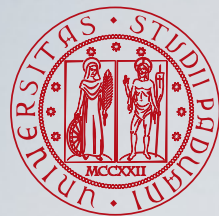


Example 1: add a new paragraph

```
var ourDiv = document.getElementById("our-div");  
var newParagraph = document.createElement("p");  
var newText = document.createTextNode("Hello, world!");  
newParagraph.appendChild(newText);  
ourDiv.appendChild(newParagraph);
```

Example 2: add a new heading

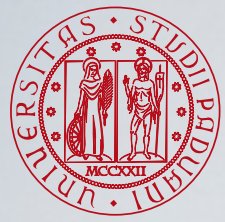
```
var ourDiv = document.getElementById("our-div");  
var para = document.getElementById("our-paragraph");  
var newHeading = document.createElement("h1");  
var headingText = document.createTextNode("A new heading");  
newHeading.appendChild(headingText);  
ourDiv.insertBefore(newHeading, para);
```

Removing and Replacing Nodes



- The **removeChild()** method removes a node from the document tree. Invoke the method on the parent node (not the node that you want to remove) and pass the child node that is to be removed as the method argument.
- The **replaceChild()** method removes one child node and replaces it with a new one. Invoke this method on the parent node, passing the new node as the first argument and the node to be replaced as the second argument.



Removing and Replacing Nodes: Examples

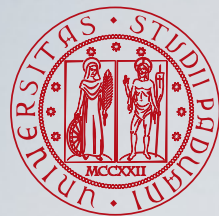
Example 1: remove an element

```
var parentDiv = document.getElementById("parent");  
var remove_el = document.getElementById("removable_element");  
parentDiv.removeChild(remove_el);
```

Example 2: replace an element with an image

```
var parentDiv = document.getElementById("parent");  
var swap_el = document.getElementById("swap-me");  
var newImg = document.createElement("img");  
newImg.setAttribute("src", "path/to/image.jpg");  
ourDiv.replaceChild(newImg, swap_el);
```

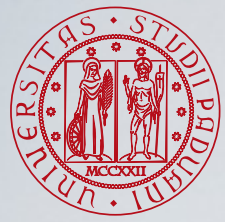
Handling Events



JavaScript Timeline

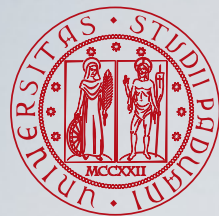
1. The web browser creates a Document object and begins parsing the web page.
2. When the HTML parser encounters `<script>` elements, it adds those elements to the document and then executes the script. These scripts are executed synchronously, and the parser pauses while the script downloads (if necessary) and runs.
3. The document is completely parsed at this point, but the browser may still be waiting for additional content, such as images, to load. When all such content finishes loading, and when all scripts have loaded and executed, the `document.readyState` property changes to **complete** and the web browser fires a load event on the Window object.
4. From this point on, event handlers are invoked asynchronously in response to user input events, network events, timer expirations, and so on.

- **Events** are occurrences that a web browser will notify your JavaScript program about.
- The web browser generates an event when:
 - it finishes loading a document;
 - the user moves the mouse over a hyperlink;
 - the user strikes a key on the keyboard;
 - . . .
- If a JavaScript application cares about a particular type of event, it can register one or more functions to be invoked when events of that type occur.



Event Type and Target

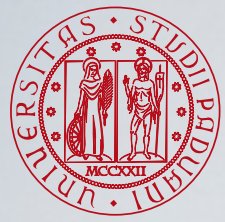
- The **event type** is a string that specifies what kind of event occurred, examples are: mouseup, keydown, click, ...
- The **event target** is the object on which the event occurred or with which the event is associated.
- When we speak of an event, we must specify both the type and the target: for example a load event on a Window, or a click event on a `<button>` Element.
- Window, Document, and Element objects are the most common event targets in client-side JavaScript applications.



Event Handler



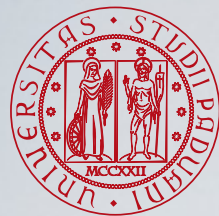
- An event handler or event listener is a function that handles or responds to an event.
- Applications register their event handler functions with the web browser, specifying an event type and an event target.
- When an event of the specified type occurs on the specified target, the browser invokes the handler.
- When event handlers are invoked for an object, we sometimes say that the browser has “fired”, “triggered”, or “dispatched” the event.



Event Object



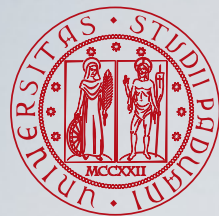
- An **event object** is an object that is associated with a particular event and contains details about that event.
- Event objects are passed as an argument to the event handler function.
- All event objects have a type property that specifies the event type and a target property that specifies the event target.
- Each event type defines a set of properties for its associated event object, for example the object associated with a mouse event includes the coordinates of the mouse pointer.



Types of Events

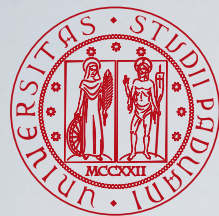
- In the early days of the Web, there was only a small set of events: “load”, “click”, and “mouseover”.
- The number of events supported by browsers has been growing rapidly, with new events coming from three sources: the DOM Level 3 events specification (<https://www.w3.org/TR/uievents/>), new APIs in the HTML5 specification, and touch-based and JavaScript-enabled mobile devices.
- Events most often used in a web apps are events for dealing with:
 - the mouse;
 - the keyboard;
 - HTML forms;
 - the Window object.

Full list of events: https://www.w3schools.com/jsref/dom_obj_event.asp



Mouse Events

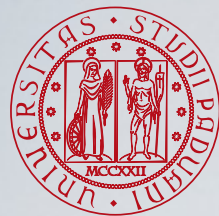
- Mouse events are generated when the user moves or clicks the mouse over a document.
- The **mousemove** event is triggered any time the user moves or drags the mouse.
- The **mousedown** and **mouseup** events are triggered when the user presses and releases a mouse button.
- The **click** event is triggered on any document element, not just form elements, when a click occurs.
- The second click event will be followed by a **dblclick** event.
- When the user moves the mouse so that it goes over a new element, the browser fires a **mouseover** event on that element. When the mouse moves so that it is no longer over an element, the browser fires a **mouseout** event on that element.
- When the user rotates the mouse wheel, browsers trigger a **mousewheel** event.



Key Events

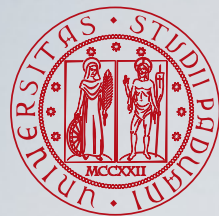
- Keyboard events are triggered on whatever document element has **keyboard focus**, and they bubble up to the document and window.
- The **keydown** and **keyup** events are low-level keyboard events: they are triggered whenever a key is pressed or released.
- When a keydown event generates a printable character, an additional **keypress** event is triggered after the keydown but before the keyup.

Practical example: <https://developer.mozilla.org/en-US/docs/Web/API/Event/type>



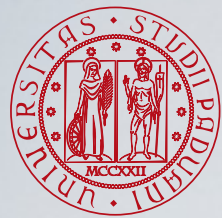
Form Events

- Form elements typically fire a click or change event when the user interacts with them, and you can handle these events by defining an **onclick** or **onchange** event handler.
- In general, form elements that are buttons fire a click event when activated (even when this activation happens through the keyboard rather than via an actual mouse click).
- Other form elements fire a change event when the user changes the value represented by the element. This happens when the user enters text in a text field or selects an option from a drop-down list. Note that this event is not fired every time the user types a key in a text field. It is fired only when the user changes the value of an element and then moves the input focus to some other form element.
- Radio buttons and checkboxes are buttons that have a state, and they fire both click and change events; the change event is the more useful of the two.
- Form elements also fire a **focus** event when they receive keyboard focus and a **blur** event when they lose it.



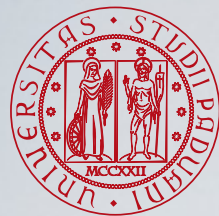
Form Events

- Each Form element has an `onsubmit` event handler to detect form submission and an `onreset` event handler to detect form resets.
- Form validation: the `onsubmit` handler is triggered just before the form is submitted by a click on a submit button; it can cancel the submission by returning `false`.
- The `onreset` event handler is invoked just before the form is reset, and it can prevent the form elements from being reset by returning `false`, it is used to make the user confirm the reset.



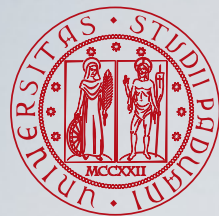
Form Events

HTML Element	Event handler
<code><input type="button"></code> or <code><button type="button"></code>	<code>onclick</code>
<code><input type="checkbox"></code>	<code>onchange, onclick</code>
<code><input type=". . . "></code> (type = text, password, file...)	<code>onchange</code>
<code><input type="radio"></code>	<code>onchange, onclick</code>
<code><input type="reset"></code> or <code><button type="reset"></code>	<code>onclick, onreset</code>
<code><select></code>	<code>onchange</code>
<code><input type="submit"></code> or <code><button type="submit"></code>	<code>onclick, onsubmit</code>
<code><textarea></code>	<code>onchange</code>



Window Events

- Window events represent occurrences related to the browser window itself.
- The **load** event is fired when a document and all of its external resources (such as images, style sheets, or scripts) are fully loaded and displayed to the user.
- The **unload** event is the opposite of load: it is triggered when the user is navigating away from a document. An unload event handler might be used to save the user's state.
- The **beforeunload** event is similar to unload but gives you the opportunity to ask the user to confirm that they really want to navigate away from your web page.
- The **resize** and **scroll** events are fired on a window when the user resizes or scrolls the browser window.

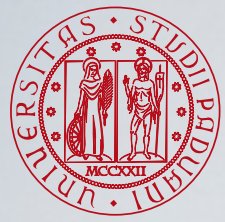


Registering Event Handlers



- There are two basic ways to register event handlers:
 - the first is to set a **property** on the object or document element that is the event target:
 - You can set an **event handler property** in JavaScript code;
 - For document elements, you can set the corresponding **attribute** directly in HTML.
 - The second, newer technique is to pass the handler to a **method** of the object or element, for handler registration:
 - there is a standard method, named **addEventListener()**;
 - a different method, named **attachEvent()**, (IE8, IE9).



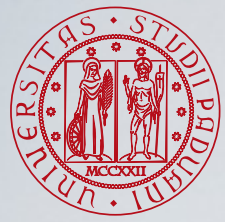


Setting Event Handlers Properties

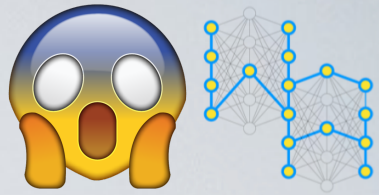


- The simplest way to register an event handler is by setting a property of the event target to the desired event handler function.
- By convention, event handler properties have names that consist of the word “on” followed by the event name: `onclick`, `onchange`, `onload`, `onmouseover`, and so on.

```
window.onload = function() {  
    var elt = document.getElementById("address");  
    elt.onsubmit = function() { return validate(this); }  
}
```



Setting Event Handler Attributes



- The event handler properties of a document element can be set as attributes on the corresponding HTML tag: the attribute value should be a string of JavaScript code (the body of the event handler function, not a complete function declaration).

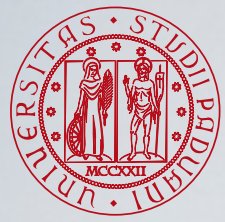
```
<button onclick="alert( 'Thank you' );">Click Here</button>
```

- To keep HTML content separate from JavaScript behavior you should avoid HTML event handler attributes.

addEventListener()

- Any object that can be an event target (includes the Window, Document and all Elements objects), defines a method named **addEventListener()** that you can use to register an event handler for that target.
- addEventListener() takes two mandatory arguments:
 - the event **type** (string without the 'on' prefix) for which the handler is being registered;
 - the **function** that should be invoked when the specified type of event occurs.

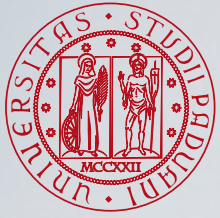
```
<button id="mybutton">Click me</button>
<script>
  var b = document.getElementById("mybutton");
  b.onclick = function() { alert("Thanks for clicking me!"); };
  b.addEventListener("click", function() { alert("Thanks again!"); });
</script>
```



Why Use `addEventListener()`?



- It allows **adding more than a single handler** for an event. This is particularly useful for AJAX libraries, JavaScript modules, or any other kind of code that needs to work well with other libraries/extensions.
- It gives you **finer-grained control** of the phase when the listener is activated (capturing vs. bubbling).
- It works on **any DOM element**, not just HTML elements.



removeEventListener()

- addEventListener() is paired with a **removeEventListener()** method which removes an event handler function from an object.
- It has the same two mandatory arguments as the addEventListener():
 - the **type**, a string which specifies the type of event for which to remove an event listener;
 - the **function** of the event handler to remove from the event target.
- It is useful to temporarily register an event handler and then remove it soon afterward.

```
document.removeEventListener("mousemove", handleMouseMove);  
document.removeEventListener("mouseup", handleMouseUp);
```




Questions?